

14. Lire un bouton

Réagir à la manette.

1. Ce qu'il faut comprendre

`bouton(A)` rend 1 tant que le bouton est enfoncé. Les huit noms existent : `A B HAUT BAS GAUCHE DROITE START SELECT`.

Ici on lit le **niveau** : tant que A est tenu, le texte change. C'est ce qu'on veut pour courir ou viser — mais pas pour un menu : c'est la leçon « le front » qui règle ce cas.

`&&`, `||` et `!` s'écrivent comme en C++, et **s'arrêtent dès que la réponse est connue** : dans `i < n && t[i] == 0`, la case n'est pas lue si l'index est hors du tableau.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **511 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage – la mise en route de la console
main()         65 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte – attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() – écrit un texte, lettre par lettre
CopierTuiles   76 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
LireManette    37 o ← bouton() – lit les huit boutons
Tuiles         160 o ← CopierTuiles – 10 lettres sur 43
"APPUIE SUR A" 12 o ← texte() – un texte gravé
"BRAVO"         " 12 o ← texte() – un texte gravé
"APPUIE SUR A" 12 o ← texte() – un texte gravé
```

Lettres gravées : ABEIOPRSUV

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

```
texte() × 3    33 o sur place, dans main() + EcrireTexte (10 o)
bouton() × 3   15 o sur place, dans main() + LireManette (37 o)
image() × 1    3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <texte>    // écrit un texte à l'écran
#include <bouton>    // lit un bouton de la manette
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    texte(3, 5, "APPUIE SUR A");

    while (true) {
        image();

        if (bouton(A) && !bouton(B)) {
            texte(3, 5, "BRAVO      ");
        }

        if (bouton(B)) {
            texte(3, 5, "APPUIE SUR A");
        }
    }

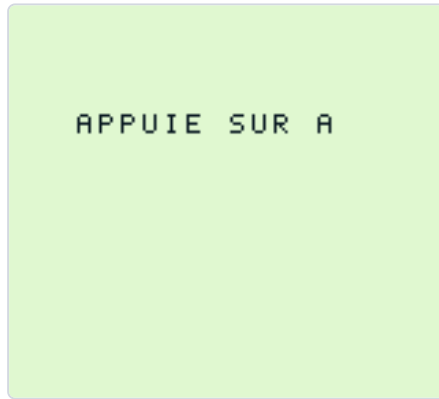
    return 0;
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.

BRAVO

A — l'écran a changé

APPUIE SUR A

B — l'écran a changé

5. Ce qu'on doit voir

« BRAVO » pendant qu'on tient A ; B remet le texte de départ.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le texte de départ est là
- ✓ A écrit « BRAVO »
- ✓ B le remet comme avant

Ensuite : **15. Un compteur qui déborde à 255** — chapitre 3 : Retenir, et réagir. · gameboy3 — écrire une cartouche Game Boy en C++